

HAB Overview

80-CG735-1 Rev. AA

Confidential and Proprietary – Qualcomm Technologies International, Ltd.

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to:
DocCtrlAgent@qualcomm.com.

Restricted Distribution: Not to be distributed to anyone who is not an employee of either Qualcomm Technologies International, Ltd. or its affiliated companies without the express approval of Qualcomm Configuration Management.

Qualcomm
Confidential - May Contain Trade Secrets
2023-02-16 02:24:03 GMT
chen@hangsheng.com.cn

Confidential and Proprietary – Qualcomm Technologies International, Ltd.

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Restricted Distribution: Not to be distributed to anyone who is not an employee of either Qualcomm Technologies International, Ltd. or its affiliated companies without the express approval of Qualcomm Configuration Management.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies International, Ltd.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark of Qualcomm Incorporated, registered in the United States and other countries. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer ("export") laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies International, Ltd. (formerly known as Cambridge Silicon Radio Limited) is a company registered in England and Wales with a registered office at: Churchill House, Cambridge Business Park, Cowley Road, Cambridge, CB4 0WZ, United Kingdom.
Registered Number: 3665875 | VAT number: GB787433096.

© 2018 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision History

Revision	Date	Description
AA	November 2018	Initial release. Alternative document number CS-00413490-DC.

Qualcomm
Confidential - May Contain Trade Secrets
2023-02-16 02:24:03 GMT
chenchen@hangsheng.com.cn

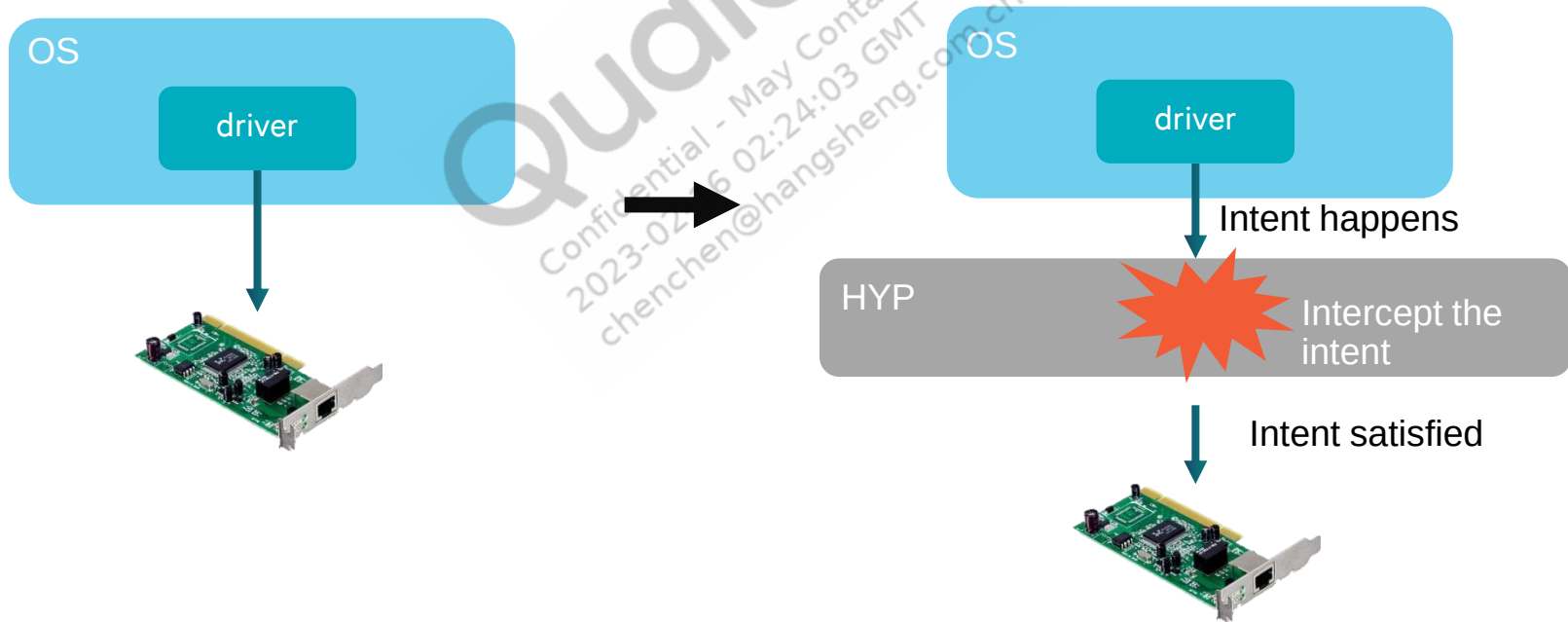
Contents

- I/O device virtualization methods and examples
 - ▣ Emulation – full virtualization
 - ▣ Passthrough – special case
 - ▣ Front/back end – para virtualization
- HAB targets, arch and APIs
 - ▣ What are HAB's goals
 - ▣ HAB's 3-layer architecture
 - ▣ What are HAB's APIs

Qualcomm
Confidential - May Contain Trade Secrets
2023-02-16 02:24:03 GMT
chenchen@hangsheng.com.cn

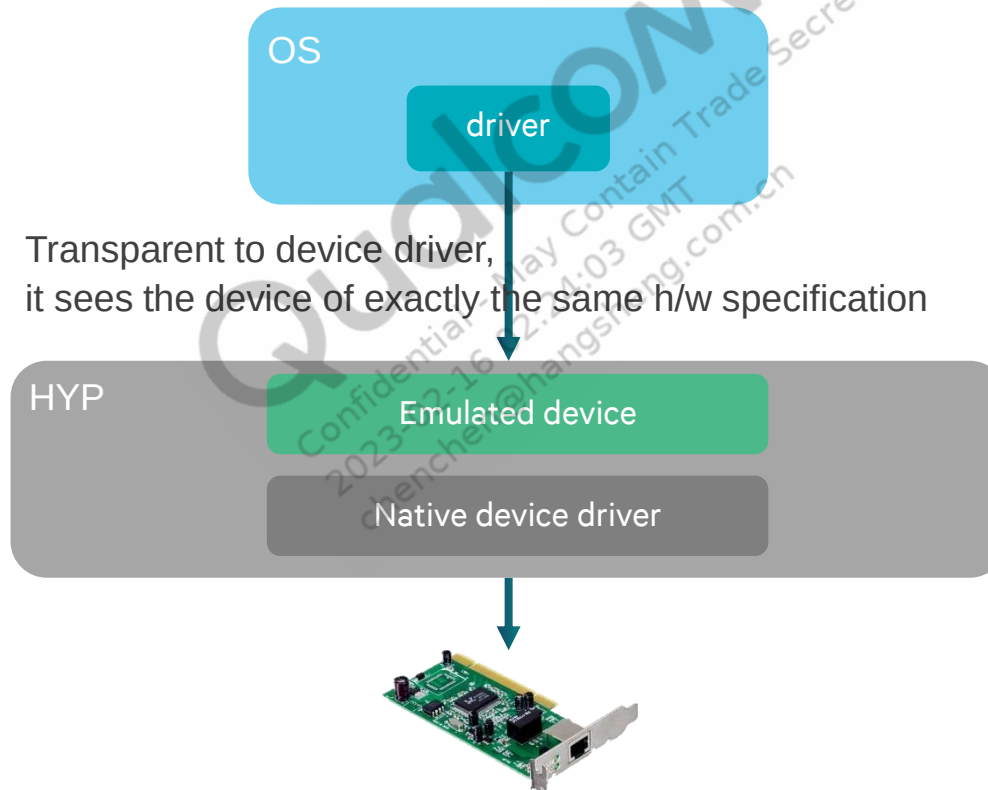
I/O device virtualization methods and examples

- Guest OS intends to use I/O device, like Ethernet, storage, audio, and display
- Hypervisor needs full control/management of the underlying h/w, including I/O devices
- Hypervisor needs to intercept the I/O device access intents from Guest OS
- Guest OS can eventually be satisfied by the Hypervisor no matter what methods are used by HYP
- Depending on different intercept-satisfying methods used, different I/O device virtualization methods come up



I/O device virtualization methods and examples

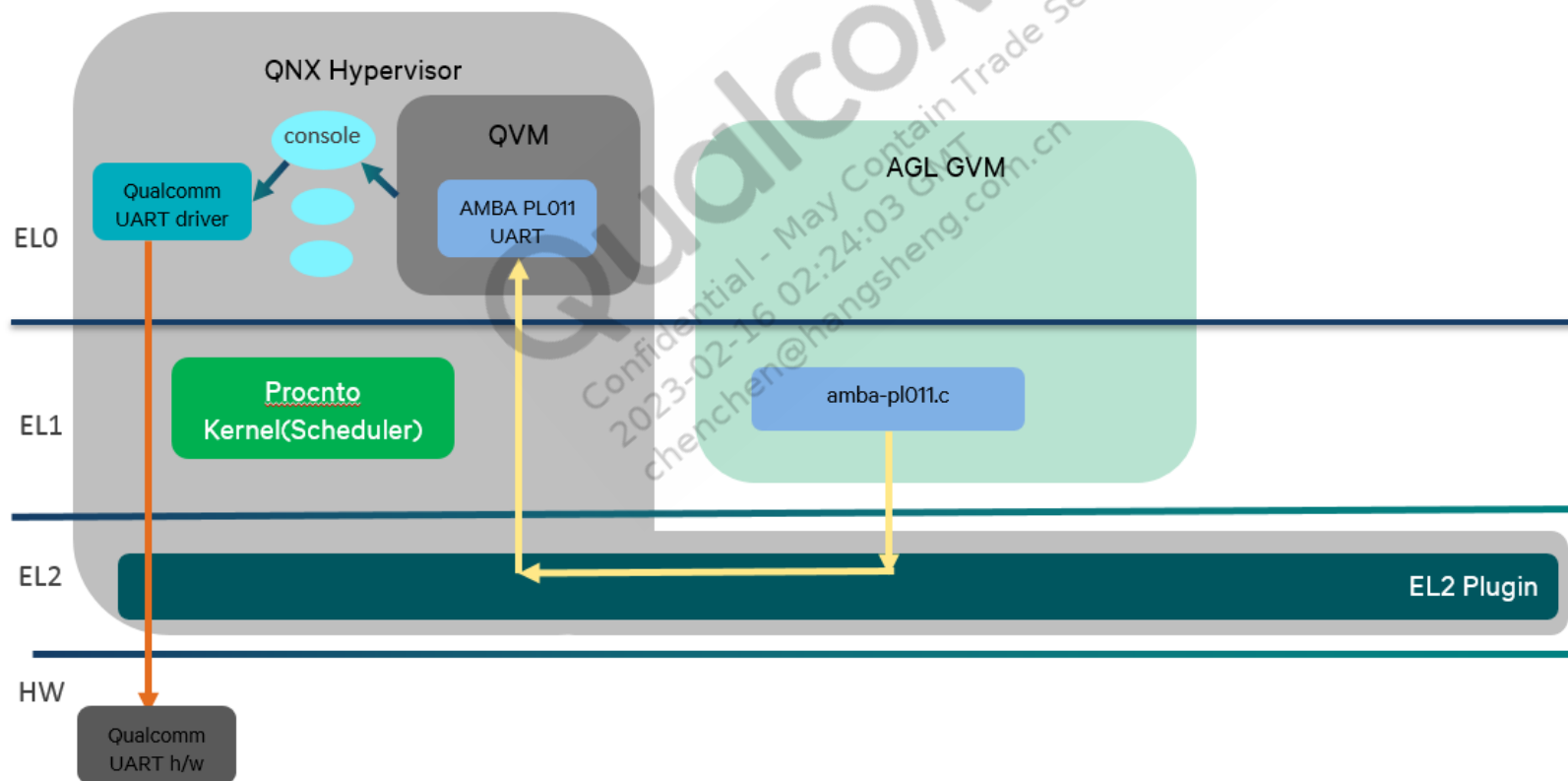
- Emulation – full virtualization
 - The exactly same device driver is used in guest OS as in native OS
 - Trap and emulate** is the key mechanism to support the device emulation



I/O device virtualization methods and examples

Emulation – full virtualization

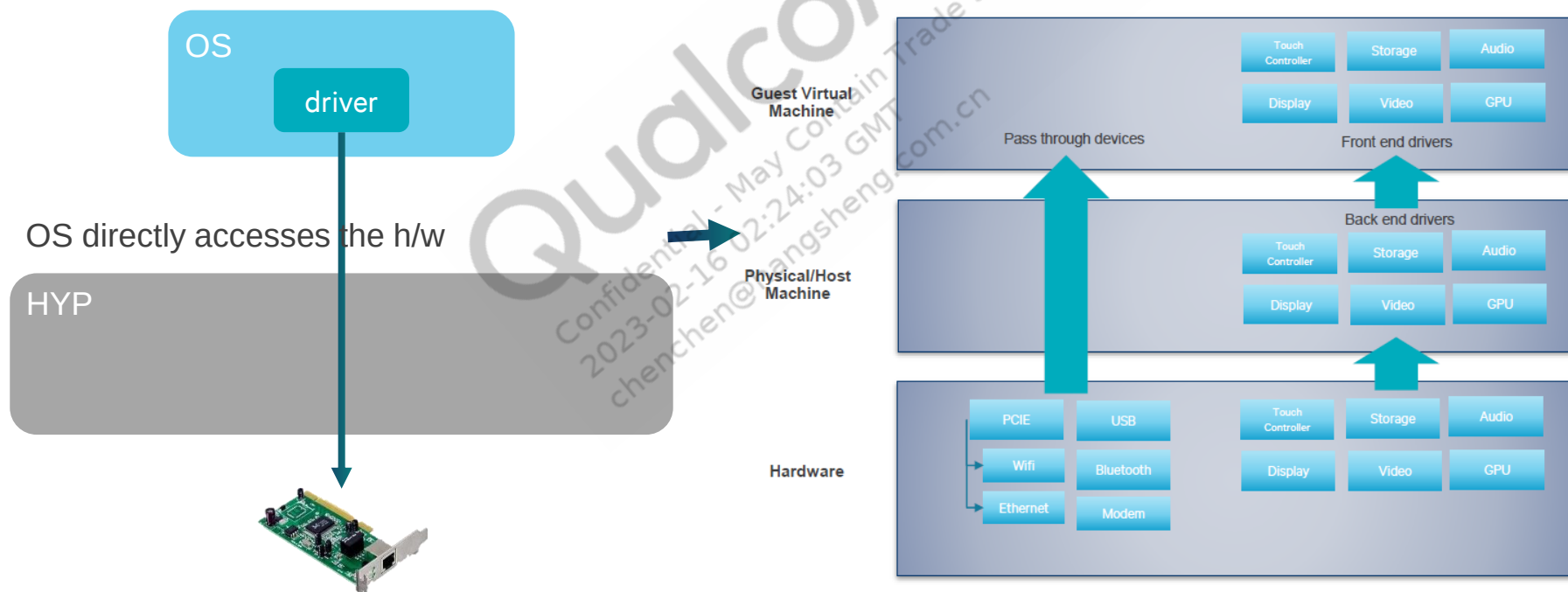
- AMBA UART is the UART IP from ARM, including AMBA PL010 and PL011
- In QNX HYP, AMBA UART device for AGL GVM is emulated by QNX Host. So, from AGL GVM AMBA UART driver perspective, it sees exactly the AMBA UART h/w interface
- This emulated UART is used for AGL GVM to output its log, like Linux kernel's log



I/O device virtualization methods and examples

- Passthrough – special case

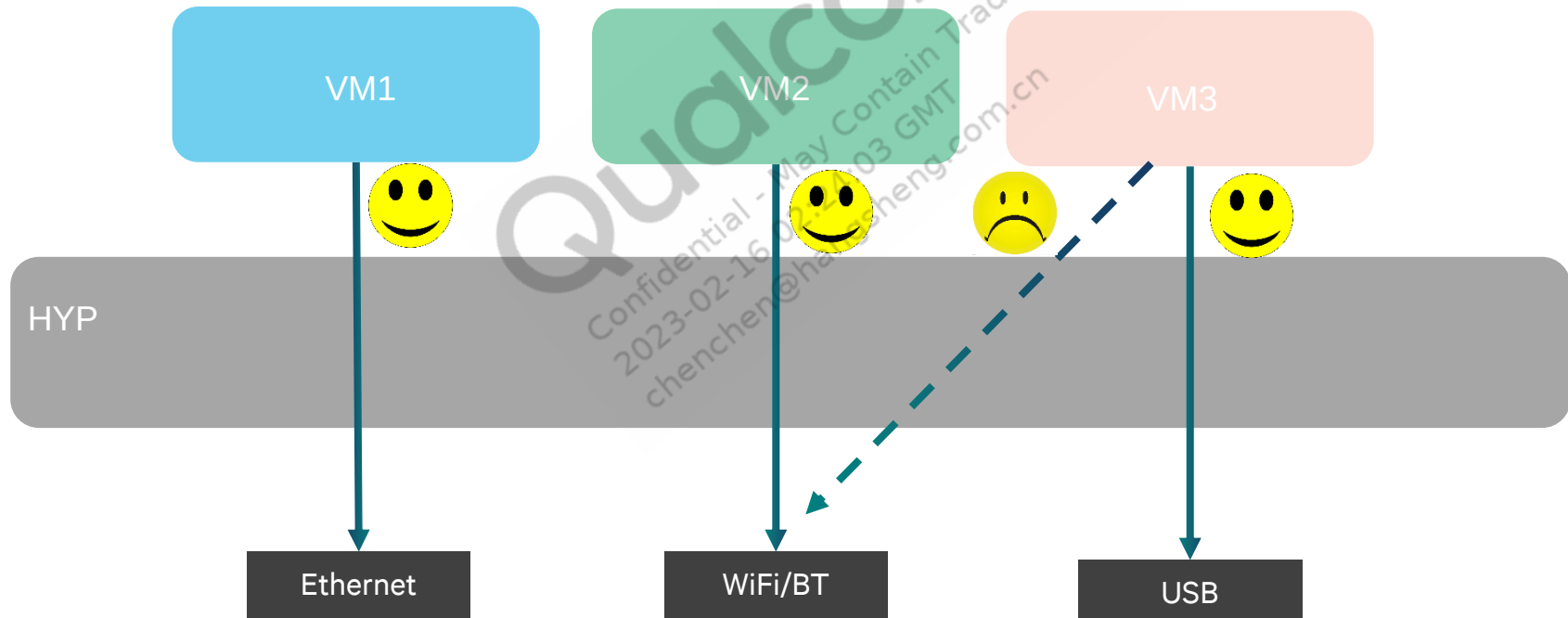
- GVM can directly use the native device driver to access the real device
- For an GVM, it can access some devices in passthrough mode while at the same time accessing other devices with pervious virtualization methods



I/O device virtualization methods and examples

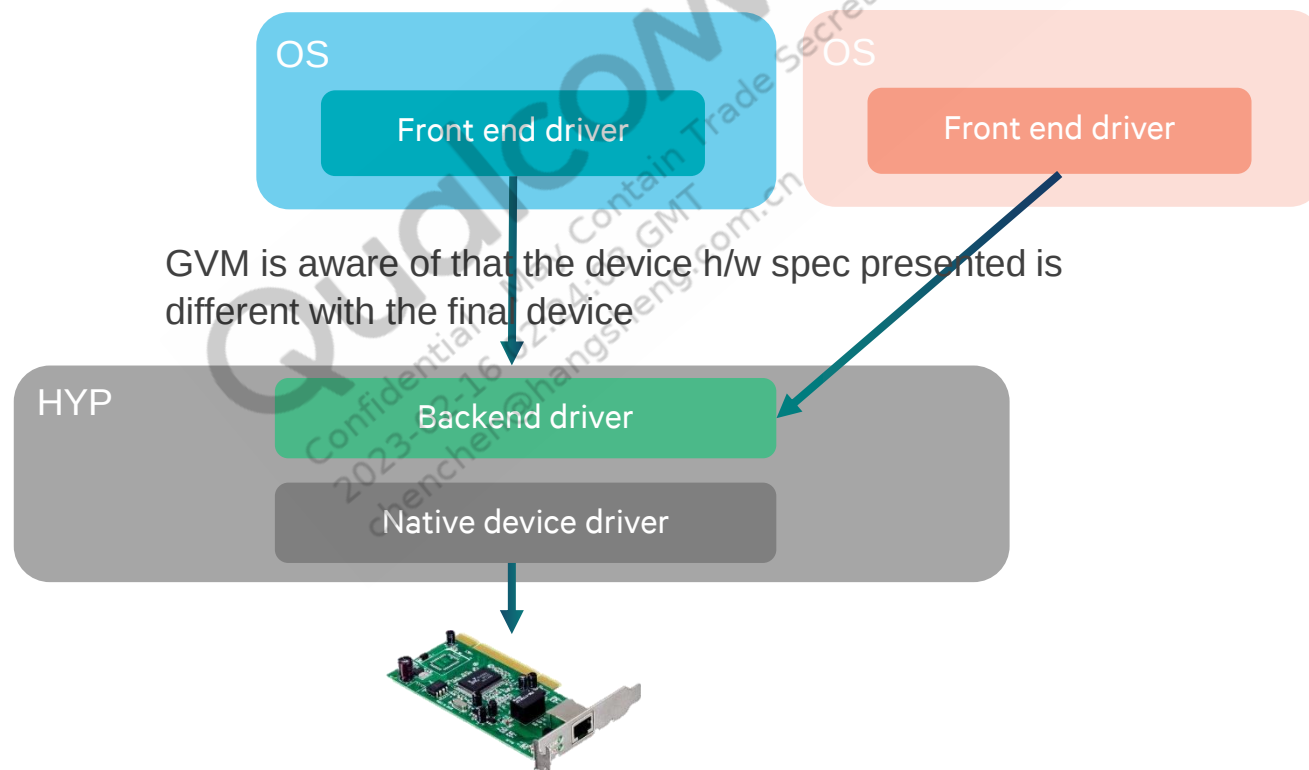
- Passthrough – special case

- The precondition is we can guarantee that the specific device is only touched by one specific VM
- That is, two VMs cannot touch the same device in pass-through mode. It is a natural result, otherwise race conflicts happen



I/O device virtualization methods and examples

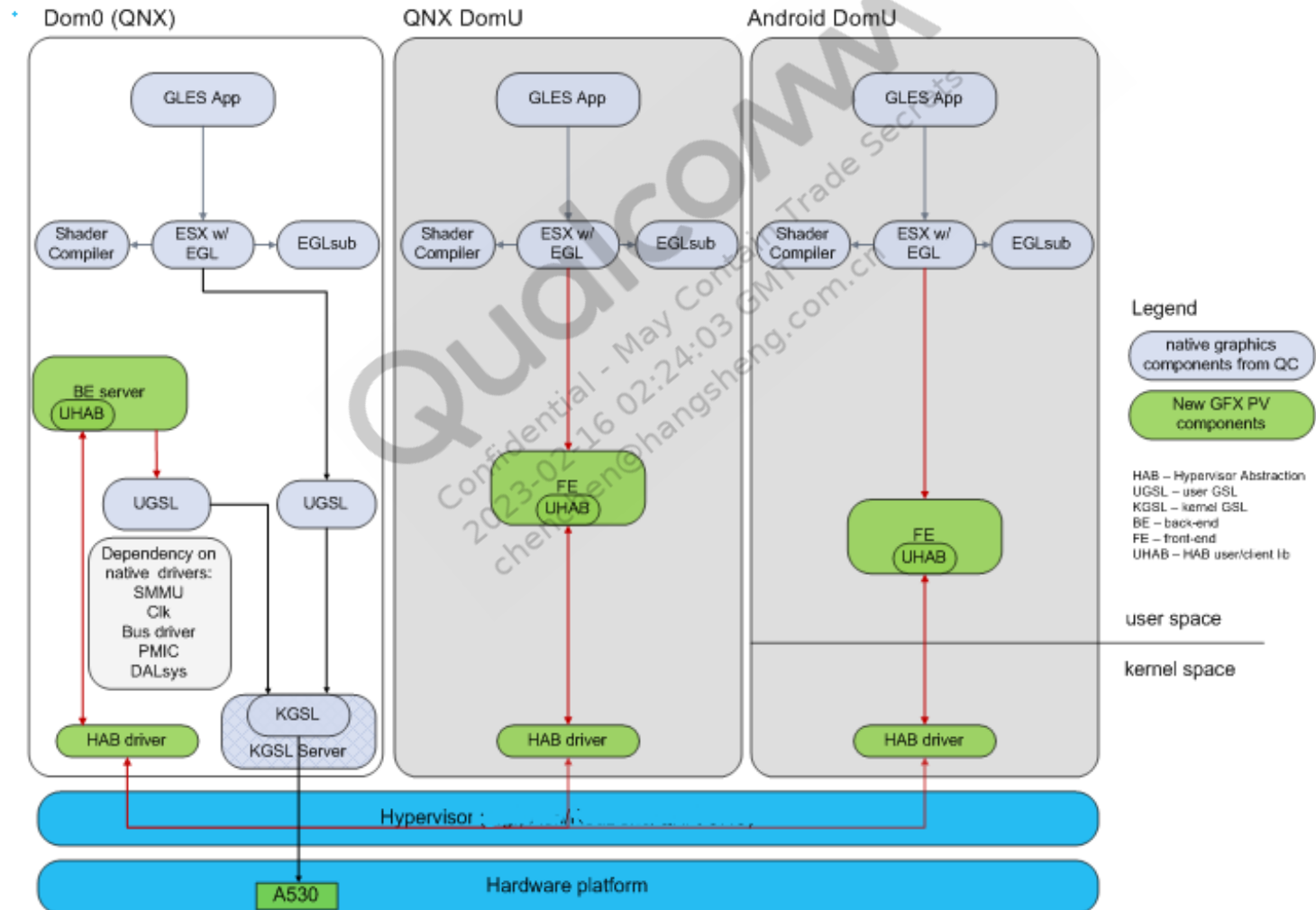
- Front and back end – para virtualization
 - Front end driver talks with back end driver
 - Then backend driver touches the real h/w through the native device driver
 - With this method, **trap and emulate** cases can be decreased so that performance is better than full virtualization



I/O device virtualization methods and examples

- Front and back end – para virtualization

- In QNX HYP, multimedia cases are of this category, like audio, video, graphics, display, and camera



HAB (Hypervior ABstraction)

Para-virtualization(FE/BE) has a performance advantage over full virtualization.

The following platform-level supports should be provided to support FE/BE virtualization:

- cross-OSes message send/recv, like between the GVM and the host OS
- cross-OSes memory sharing

Hypervisor should have a fundamental mechanism; for example, the virtual shm device supports cross-OSes communication in QNX HYP.

However, different hypervisors (QNX, GreenHill, and OpenSynergy) have different mechanisms.

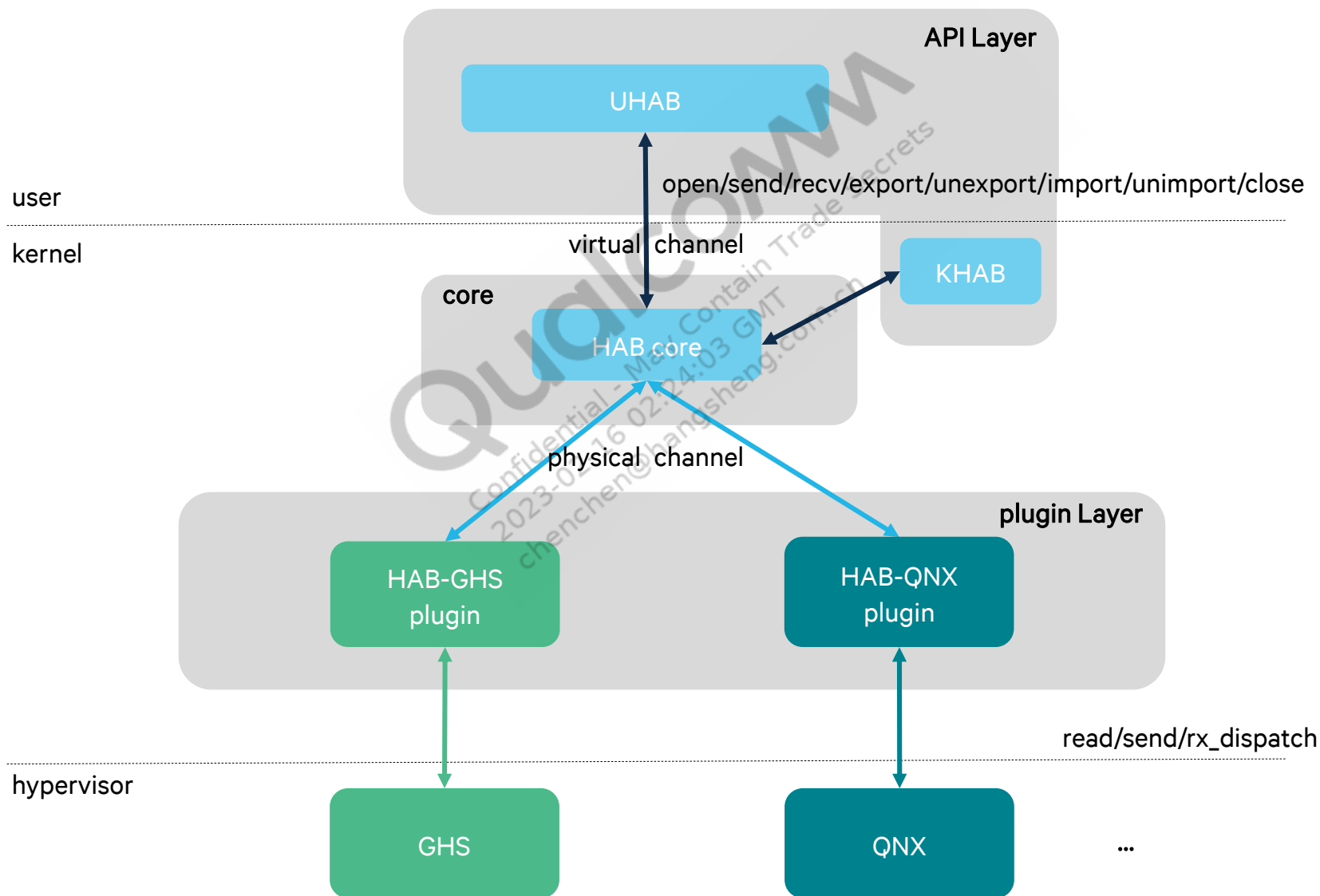
Therefore, HAB comes up and sets the following goals:

- Supports cross-OSes message send/recv
- Supports cross-OSes memory sharing
- Provides the same APIs throughout all the HYPs
- Provides the same APIs throughout all the OSes, including Linux and QNX
- Supports both kernel and user land if there is user/kernel land variants in a given OS
- Guarantees any security requirement among processes and OSes, like one OS is not allowed to sniff communications between other two OSes

Note that HAB's original goal is to provide the connection/bridge/channel between FE and BE. FE acts as a client while BE as a server. This helps understand HAB's internal design.

HAB 3-layers arch

- 3 layers: API layer, core layer, and HAB-HYP plugin layer



HAB APIs

- 2 sets with different purposes: communication and memory sharing APIs
- Uhab and khab: APIs for both user and kernel space front/back ends
- Communication APIs (one to one)
- MMIDs are used to identify different physical channels that are pre-defined and used by different functions/models such as MM_AUD_1, MM_CAM_1, and MM_DISP_1

```
int32_t habmm_socket_open(int32_t *handle, enum MM_ID mm_ip_id, uint32_t timeout, uint32_t flags);  
int32_t habmm_socket_close(int32_t handle);  
int32_t habmm_socket_send(int32_t handle, void *src_buff, uint32_t size_bytes, uint32_t flags);  
int32_t habmm_socket_recv(int32_t handle, void *dst_buff, uint32_t *size_bytes, uint32_t timeout, uint32_t flags);
```

HAB APIs

- Memory sharing APIs (local and remote)

```
int32_t habmm_export(int32_t handle, void *buff_to_share, uint32_t size_bytes, uint32_t *export_id, uint32_t flags);  
int32_t habmm_unexport(int32_t handle, uint32_t export_id, void *buff_to_share, uint32_t flags);  
int32_t habmm_import(int32_t handle, void **buff_shared, uint32_t size_bytes, uint32_t export_id, uint32_t flags);  
int32_t habmm_unimport(int32_t handle, uint32_t export_id, void *buff_shared, uint32_t flags);
```

Questions?

For additional information or to submit technical questions, go to:

<https://createpoint.qti.qualcomm.com>

Thank You

Qualcomm
Confidential - May Contain Trade Secrets
2023-02-16 02:24:03 GMT
chenchen@hangsheng.com.cn